

第7章

数字签名

数字签名在身份认证、数据完整性、不可否认性以及匿名性等信息安全领域中有重要应用,特别是在大型网络安全通信中的密钥分配、认证,以及电子商务系统中发挥着重要作用。数字签名是实现认证的重要工具。本章介绍数字签名的基本概念,以及各种常用的数字签名体制,如 RSA, ElGamal, Schnorr, DSS 和 SM2 等签名体制。此外,还要介绍一些特殊用途的数字签名,如不可否认签名、防失败签名、盲签名和群签名等。

7.1

数字签名基本概念

政治、军事、外交等文件、命令和条约,商业契约以及个人书信等,传统上采用手书签名或印章,以便在法律上能认证、核准、生效。随着计算机通信网的发展,人们希望通过电子设备实现快速、远距离的交易,数字(或电子)签名法应运而生,并开始用于商业通信系统,如电子邮递、电子转账和办公自动化等系统。

类似于手书签名,数字签名也应满足以下要求:

- (1) 收方能够确认或证实发方的签名,但不能伪造,简记为 R1-条件。
- (2) 发方发出签名的消息给收方后,就不能再否认他所签发的消息,简记为 S-条件。
- (3) 收方对已收到的签名消息不能否认,即有收报认证,简记为 R2-条件。
- (4) 第三者可以确认收发双方之间的消息传送,但不能伪造这一过程,简记为 T-条件。

数字签名与手书签名的区别在于,手书签名是模拟的,且因人而异。数字签名是 0 和 1 的数字串,因消息而异。数字签名与消息认证的区别在于,消息认证使收方能验证消息发送者及所发消息内容是否被篡改过。当收、发者之间没有利害冲突时,这对于防止第三者的破坏来说是足够了。但当收者和发者之间有利害冲突时,单纯用消息认证技术就无法解决他们之间的纠纷,此时须借助满足前述要求的数字签名技术。

为了实现签名目的,发方必须向收方提供足够的非保密信息,以便使其能验证消息的签名;但又不能泄漏用于产生签名的机密信息,以防他人伪造签名。因此,签名者和证实者可公用的信息不能太多。任何一种产生签名的算法或函数都应当提供这两种信息,而且从公开的信息很难推测出用于产生签名的机密信息。此外,任何一种数字的实现都有赖于仔细设计的通信协议。

数字签名有两种:一种是对整体消息的签名,它是消息经过密码变换的被签消息整体;一种是对压缩消息的签名,它是附加在被签名消息之后或某一特定位置上的一段签

名图样。若按明、密文的对应关系划分，每种又可分为两个子类：一类是确定性（deterministic）数字签名，其明文与密文一一对应，它对一特定消息的签名不变化，如 RSA 和 Rabin 等签名；另一类是随机化（randomized）或概率式数字签名，它对同一消息的签名是随机变化的，取决于签名算法中的随机参数的取值。一个明文可能有多个合法数字签名，如 ElGamal 等签名。

一个签名体制一般含有两个组成部分：签名算法（Signature Algorithm）和验证算法（Verification Algorithm）。对 m 的签名可简记为 $\text{Sig}(m) = \sigma'$ ，而对 σ' 的验证简记为 $\text{Ver}(\sigma') = \{\text{真}, \text{伪}\} = \{0, 1\}$ 。签名算法或签名密钥是秘密的，只有签名人掌握；验证算法应当公开，以便于他人进行验证。

一个签名体制可由量 (M, S, K, V) 组成，其中 M 是明文空间， S 是签名的集合， K 是密钥空间， V 是验证函数的值域，由真、伪组成。

对于每一个 $k \in K$ 有一签名算法，易于计算：

$$\sigma' = \text{Sig}_k(m) \in S \quad (7-1)$$

和一验证算法：

$$\text{Ver}_k(m, \sigma') \in \{\text{真}, \text{伪}\} \quad (7-2)$$

它们对每一 $m \in M$ ，有签名 $\text{Sig}_k(m) \in S$ （为 $M \rightarrow S$ 的映射）。 (m, σ') 对易于验证 S 是否为 m 的签名：

$$\text{Ver}_k(m, \sigma') = \begin{cases} \text{真}, & \text{当 } \sigma = \text{Sig}(m) \\ \text{伪}, & \text{当 } \sigma \neq \text{Sig}(m) \end{cases} \quad (7-3)$$

体制的安全性在于，从 m 和其签名 σ' 难于推出 k 或伪造一个 m' ，使 m' 和 σ' 可被证实为真。

消息签名与消息加密有所不同。消息加密和解密可能是一次性的，它要求在解密之前是安全的；而一个签名的消息可能作为一个法律上的文件，如合同等，很可能在对消息签署多年之后才验证其签名，且可能需要多次验证此签名。因此，人们对签名的安全性和防伪造的要求更高，且要求证实速度比签名速度更快，特别是联机在线实时验证。

随着计算机网络的发展，过去依赖于手书签名的各种业务都可用这种电子数字签名代替，它是实现电子贸易、电子支票、电子货币、电子购物、电子出版及知识产权保护等系统安全的重要保证。有关签名算法的综合性介绍可参阅相关文献[Diffie 等 1976; Menezes 等 1997; Mitchell 等 1992; Schneier 1996; Stinson 1995; Rivest 1990b]。

7.2

RSA 签名体制

7.2.1 体制参数

令 $n = p_1 p_2$ ， p_1 和 p_2 是大素数，令 $M = S = Z_n$ ，选 e 并计算出 d 使 $ed \equiv 1 \pmod{\phi(n)}$ ，公开 n 和 e ，将 p_1 、 p_2 和 d 保密。 $K = (n, p, q, e, d)$ 。

7.2.2 签名过程

对消息 $M \in Z_n$ ，定义：

$$S = \text{Sig}_k(M) = M^d \bmod n \quad (7-4)$$

为对 M 的签名。

7.2.3 验证过程

对给定的 M 和 S ，可按式 (7-5) 验证：

$$\text{Ver}_k(M, S) = \text{真} \Leftrightarrow M \equiv S^e \bmod n \quad (7-5)$$

7.2.4 安全性

显然，由于只有签名者知道 d ，根据 RSA 体制知道其他人不能伪造签名，但易于证实所给任意 (M, S) 对是否由消息 M 和相应签名构成的合法对。如第 5 章中所述，RSA 体制的安全性依赖于 $n = p_1 p_2$ 分解的困难性[Rivest 1978]。

ISO/IEC 9796 和 ANSI X9.30-199X 已将 RSA 作为建议数字签名标准算法[Menezes 等 1997]。PKCS #1 是一种采用杂凑算法（如 MD-2 或 MD-5 等）和 RSA 相结合的公钥密码标准[RSA Lab 1993; Menezes 等 1997]。有关 ISO/IEC 9796 安全性分析可参阅相关文献[Guillou 等 1990]。

7.3

ElGamal 签名体制

ElGamal 签名体制由 T. ElGamal 于 1985 年提出，其修正形式已被美国 NIST 作为数字签名标准（DSS）。它是 Rabin 体制的一种变型，专门设计作为签名用。方案的安全性基于求离散对数的困难性。它是一种非确定性的双钥体制，即对同一明文消息，由于随机参数选择不同而有不同的签名。

7.3.1 体制参数

p : 一个大素数，可使 Z_p 中求解离散对数为困难问题；

g : 是 Z_p 中乘群 Z_p^* 的一个生成元或本原元素；

M : 消息空间，为 Z_p^* ；

S : 签名空间，为 $Z_p^* \times Z_{p-1}$ ；

x : 用户密钥 $x \in Z_p^*$ 。

$$y = g^x \bmod p \quad (7-6)$$

密钥： $K = (p, q, x, y)$ ，其中 p ， g 和 y 为公钥， x 为密钥。

7.3.2 签名过程

给定消息 M ，发端用户进行下述工作：

(1) 选择秘密随机数 $k \in Z_p^*$ 。

(2) 计算 $H(M)$ ：

$$r = g^k \bmod p \quad (7-7)$$

$$s = (H(M) - xr)k^{-1} \bmod (p-1) \quad (7-8)$$

(3) 将 $\text{Sig}_k(M, k) = S = (r \| s)$ 作为签名，将 M 和 $(r \| s)$ 送给对方。

7.3.3 验证过程

收信人收到 $M, (r \| s)$ ，先计算 $H(M)$ ，并按式 (7-9) 验证：

$$\text{Ver}_k(H(M), r, s) = \text{真} \Leftrightarrow y^r r^s \equiv g^{H(M)} \bmod p \quad (7-9)$$

这是因为 $y^r r^s \equiv g^{rx} g^{sk} \equiv g^{(rx+sk)} \bmod p$ ，由式 (7-8) 有：

$$(rs+sk) \equiv H(M) \bmod (p-1) \quad (7-10)$$

故有：

$$y^r r^s \equiv g^{H(M)} \bmod p \quad (7-11)$$

在此方案中，对同一消息 M ，由于随机数 k 不同而有不同的签名值 $S = (r \| s)$ 。

例 7-1 选 $p = 467$ ， $g = 2$ ， $x = 127$ ，则有 $y = g^x = 2^{127} \equiv 132 \bmod 467$ 。

若待发送消息为 M ，其杂凑值为 $H(M) = 100$ ，选随机数 $k = 213$ ，注意， $(213, 466) = 1$ 且 $213^{-1} \bmod 466 = 431$ ，则有 $r \equiv 2^{213} \equiv 29 \bmod 467$ 。 $s \equiv (100 - 127 \times 29) \times 431 \equiv 51 \bmod 466$ 。

验证：收信人先算出 $H(M) = 100$ ，然后验证 $132^{29} 29^{51} \equiv 189 \bmod 467$ ， $2^{100} \equiv 189 \bmod 467$ 。

7.3.4 安全性

(1) 不知消息签名对攻击。攻击者在不知道用户密钥 x 的情况下，若想伪造用户的签名，可选 r 的一个值，然后试验相应 s 取值，为此必须计算 $\log_g g^x s^{-r}$ 。也可先选一个 s 的取值，然后求出相应 r 的取值，试验在不知道 r 条件下分解方程：

$$y^r s^s ab \equiv g^M \bmod p$$

这些都是离散对数问题。至于能否同时选出 a 和 b ，然后解出相应 M ，这仍面临求离散对数问题，即需计算 $\log_g y^r r^s$ 。

(2) 已知消息签名对攻击。假定攻击者已知 $(r \| s)$ 是消息 M 的合法签名。令 h, i, j 是整数，其中 $h \geq 0$ ， $i, j \leq p-2$ ，且 $(hr - js, p-1) = 1$ 。攻击者可计算

$$r' = r^h y^i \bmod p \quad (7-12)$$

$$s' = s \lambda (hr - js)^{-1} \bmod (p-1) \quad (7-13)$$

$$M' = \lambda (hM + is)(hr - js)^{-1} \bmod (p-1) \quad (7-14)$$

则 $(r' \| s')$ 是消息 M' 的合法签名。但这里的消息是 M' ，并非是攻击者选择的利于他的消

息。如果攻击者要对其选定的消息得到相应的合法签名, 仍然面临求离散对数的问题。如果攻击者掌握了同一随机数 r 下的两个消息 M_1 和 M_2 的合法签名 $(r_1 \parallel b_1)$ 和 $(r_2 \parallel b_2)$, 则由:

$$M_1 = r_1 k + s_1 r \bmod (p-1) \quad (7-15)$$

$$M_2 = r_2 k + s_2 r \bmod (p-1) \quad (7-16)$$

就可以解出用户的密钥 k 。因此在实用中, 对每个消息的签名都应变换随机数 k , 而且对某消息 M 签名所用的随机数 k 不能泄漏, 否则将可由式 (7-10) 解出用户的密钥 x 。目前, ANSI X9.30-199X 已将 ElGamal 签名体制作为签名标准算法。

7.4

Schnorr 签名体制

Schnorr C 于 1989 年提出一种签名体制——Schnorr 签名体制。

7.4.1 体制参数

p, q : 大素数, $q \mid p-1$, q 是大于等于 160b 的整数, p 是大于等于 512b 的整数, 保证 Z_p 中求解离散对数困难;

g : Z_p^* 中元素, 且 $g^q \equiv 1 \bmod p$;

x : 用户密钥, $1 < x < q$;

y : 用户公钥, $y \equiv g^x \bmod p$;

消息空间 $m = Z_p^*$, 签名空间 $s = Z_p^* \times Z_q$; 密钥空间

$$k = \{(p, q, g, x, y) : y \equiv g^x \bmod p\} \quad (7-17)$$

7.4.2 签名过程

令待签消息为 M , 对给定的 M 做下述运算:

(1) 签名用户任选一秘密随机数 $k \in Z_q$ 。

(2) 计算:

$$r \equiv g^k \bmod p \quad (7-18)$$

$$s \equiv k + xe \bmod p \quad (7-19)$$

式中:

$$e = H(r \parallel M) \quad (7-20)$$

(3) 将消息 M 及其签名 $S = \text{Sig}_k(M) = (e \parallel s)$ 送给收信人。

7.4.3 验证过程

收信人收到消息 M 及签名 $S = (e \parallel s)$ 后:

(1) 计算

$$r' \equiv g^s y^e \pmod{p} \quad (7-21)$$

然后计算 $H(r' \| M)$ 。

(2) 验证

$$\text{Ver}(M, r, s) \Leftrightarrow H(r' \| M) = e \quad (7-22)$$

因为，若 $(e \| s)$ 是 M 的合法签名，则有 $g^s y^e \equiv g^{k-xe} g^{xe} \equiv g^k \equiv r \pmod{p}$ ，式 (7-22) 必成立。

7.4.4 Schnorr 签名与 ElGamal 签名的不同点

(1) 在 ElGamal 体制中， g 为 Z_p 的本原元素；在 Schnorr 体制中， g 为 Z_p^* 中子集 Z_q^* 的本原元素，它不是 Z_q^* 的本原元素。显然 ElGamal 的安全性要高于 Schnorr。

有关 Schnorr 签名的各种变型可参阅相关文献[Brickell 等 1992]。De Rooij[1991, 1993]对 Schnorr 方案的安全性进行了分析。

(2) Schnorr 的签名较短，由 $|q|$ 及 $|H(M)|$ 决定。

(3) 在 Schnorr 签名中， $r = g^k \pmod{p}$ 可以预先计算， k 与 M 无关，因而签名只需一次 $\pmod{q}$ 乘法及减法。所需计算量少、速度快，适用于智能卡应用。

例 7-2 选取素数 $q=101$ ， $p=7879(q|p-1)$ ，生成元 $g=170$ ，选取私钥 $x=75$ ，计算公钥 $y=170^{75} \pmod{7879}=4567$ ，设选定的哈希函数为 H ， A 的公开参数为 $(7879, 101, 170, 4567)$ 及 H ，私钥为 75。假设要签名的消息为 m ，签名如下：

(1) 用户 A 选取随机数 $k=50$ ，计算 $r = g^k \pmod{p} = 170^{50} \pmod{7879} = 2518$ 。

(2) 计算 $e = H(m \| r) = H(m \| 2518)$ ，假设计算的结果为 96（依赖于所选取的哈希函数）。

(3) 计算 $s = 50 + 75 \times 96 \pmod{101} = 79$ 。

(4) 签名结果为 $(e, s) = (96, 97)$ 。

签名验证：计算 $r' = 170^{79} \times 4567^{-96} \pmod{7879} = 2518$ ；检查等式 $e' = H(m \| r') = e$ 是否成立。如果相等，则接受签名。

7.5 DSS 签名标准

7.5.1 概况

DSS 签名标准是 1991 年 8 月由美国 NIST 提出，1994 年 5 月 19 日正式公布，1994 年 12 月 1 日正式采用的美国联邦信息处理标准。其中采用了第 6 章中介绍的 SHA，其安全性基于求离散对数的困难性。SHA 是在 ElGamal 和 Schnorr[1991]两个方案基础上设计的。DSS (Digital Signature Standard) 中采用的算法简记为 DSA (Digital Signature Algorithm)。此算法由 D. W. Kravitz 设计。

这类签名标准具有较好的兼容性和适用性，已成为网络安全体系的基本构件之一。

7.5.2 签名和验证签名的基本框图

图 7-1 (a) 和 (b) 分别示出了 RSA (LUC) 签名体制和 DSS 签名体制的基本框图。其中, h : hash 运算; M : 消息; E : 加密; D : 解密; K_{US} : 用户密钥; K_{UP} : 用户公钥; K_{UG} : 部分或全局用户公钥; k : 随机数。

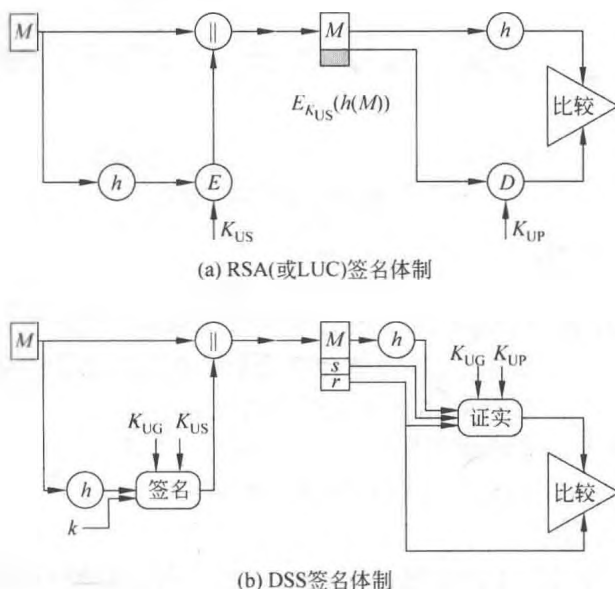


图 7-1 RSA 和 DSS 签名体制示意图

7.5.3 算法描述

(1) 全局公钥 (p, q, g) :

p : 是 $2L-1 < p < 2L$ 中的大素数, $512 \leq L \leq 1024$, 按 64b 递增;

q : $(p-1)$ 的素因子, 且 $2^{159} < q < 2^{160}$, 即字长 160b;

g : $g = h^{p-1} \bmod p$, 且 $1 < h < (p-1)$, 满足 $h^{(p-1)/q} \bmod p > 1$ 。

(2) 用户密钥 x : x 为在 $0 < x < q$ 内的随机或拟随机数。

(3) 用户公钥 y : $y = g^x \bmod p$ 。

(4) 用户对每个消息用的秘密随机数 k : 在 $0 < k < q$ 内的随机或拟随机数。

(5) 签名过程: 对消息 $M \in Z_p^*$, 其签名为

$$S = \text{Sig}_k(M, k) = (r, s) \quad (7-23)$$

其中, $S \in Z_q \times Z_q$,

$$r \equiv (g^k \bmod p) \bmod q \quad (7-24)$$

$$s \equiv [k^{-1}(h(M) + xr)] \bmod q \quad (7-25)$$

(6) 验证过程: 计算

$$w = s^{-1} \bmod q; \quad u_1 = [h(M)w] \bmod q;$$

$$u_2 = rw \bmod q; \quad v = [(g^{u_1} y^{u_2}) \bmod p] \bmod q$$

$$\text{Ver}(M, r, s) = \text{真} \Leftrightarrow v = r \quad (7-26)$$

7.5.4 DSS 签名和验证框图

图 7-2 为 DSS 签名和验证框图。

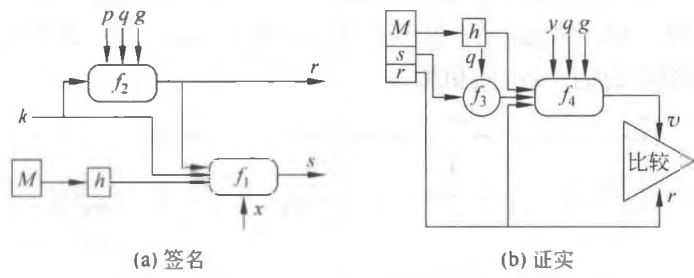


图 7-2 DSS 签名和验证框图

7.5.5 公众反应

RSA Data Security Inc (DSI) 想以 RSA 算法作为标准，因而它对此反应强烈。在标准公布之前，它就指出，采用公用模可能使政府能够伪造签名。许多大的软件公司早已得到 RSA 的许可证，从而反对 DSS。主要批评意见如下：

- (1) DSA 不能用于加密或密钥分配。
- (2) DSA 是由 NSA 开发的，算法中可能设有陷阱。
- (3) DSA 比 RSA 慢。
- (4) RSA 已是一个实际上的标准，而 DSS 与现行国际标准不相容。
- (5) DSA 未经公开选择过程，还没有足够的时间进行分析证明。
- (6) DSA 可能侵犯了其他专利（如 Schnorr 签名算法和 Diffie-Hellman 的公钥密钥分配算法）。
- (7) 由 512b 所限定的密钥量太小，现已改为凡是 512~1024b 中可被 64 除尽的数，均可供使用。有关批评意见可参阅相关文献[Smid 等 1992a]。

7.5.6 实现速度

预计算：随机数 r 与消息无关，选一数串 k ，预先计算出其 r 。对 k^{-1} 也可这样做。预计算大大加快了 DSA 的速度。DSA 和 RSA 的比较如表 7-1 所示。

表 7-1 DSA 和 RSA 比较

	DSA	RSA	DSA 采用公用 p 、 q 、 g
总计算	Off Card(P)	N/A	Off Card(P)
密钥生成	14s	Off Card(S)	4s
预计算	14s	N/A	4s
签字	0.035s	15s	0.035s
证实	16s	1.5s	10s

注意：脱卡（Off Card）计算以 33MHz 的 80386 PC， S 是脱卡秘密参数，模皆为 512b。

NIST 曾给出一种求 DSA 体制所需素数的建议算法，这一体制是在 ElGamal 体制基

础上构造的。有关 ElGamal 体制安全性讨论也涉及 DSA, 如秘密随机数 k 若被重复使用, 则有被破译的危险性。大范围用户采用同一公共模会成为众矢之的。Simmons[1993a, 1993b]还发现, DSA 可能会提供一个潜信道。还有人提出对 DSA 的各种修正方案[Yen 1994; Nyberg 等 1993]。

7.6

中国商用数字签名算法 SM2

SM2 椭圆曲线数字签名算法是 2010 年 12 月由我国国家密码管理局正式公布的商用数字签名标准。同时公布的还包含加解密算法和密钥交换协议。该组椭圆曲线密码算法已经广泛应用在多类商用密码产品之中。本节仅介绍数字签名算法, 其他算法在相关章节中介绍, 全面详细的介绍请参见 SM2 标准。

7.6.1 体制参数

(1) 选择一个椭圆曲线。国家密码管理局在 SM2 椭圆曲线公钥密码算法中推荐使用的曲线为 256 位素数域 $GF(p)$ 上的椭圆曲线。方程形式为: $y^2 = x^3 + ax + b$ 。具体曲线参数可见本书 5.6.1 节内容。

(2) 设置用户 A 的私钥 $d_A \in [1, n-1]$ 和用户 A 的公钥 $P_A = [d_A]G = (x_A, y_A)$ 。

(3) 选择一个密码杂凑算法, 设为 $H_v()$ 。表示摘要长度为 v 比特的密码杂凑函数。如国家密码管理局发布的 SM3 算法。

(4) 选择一个安全的随机数发生器, 建议选用国家密码管理局批准的随机数发生器。

(5) 假设签名者 A 具有长度为 $entlen_A$ 比特的可辨别标识 ID_A , 记 $ENTL_A$ 是由整数 $entlen_A$ 转换而成的两个字节。在椭圆曲线数字签名算法中, 签名者和验证者都需要用密码杂凑函数求得用户 A 的杂凑值 $Z_A = H_{256}(ENTL_A || ID_A || a || b || x_G || y_G || x_A || y_A)$ 。

7.6.2 签名过程

假设待签名的消息为 M , 为了获取消息 M 的数字签名 (r, s) , 作为签名者的用户 A 应执行以下运算步骤:

- (1) 置 $\bar{M} = Z_A || M$;
- (2) 计算 $e = H_v(\bar{M})$, 并将 e 的数据类型转换为整数;
- (3) 用随机数发生器产生随机数 $k \in [1, n-1]$;
- (4) 计算椭圆曲线点 $(x_1, y_1) = [k]G$, 并将 x_1 的数据类型转换为整数;
- (5) 计算 $r = (e + x_1) \bmod n$, 若 $r = 0$ 或 $r + k = n$ 则返回步骤 (3);
- (6) 计算 $s = ((1 + d_A)^{-1} \cdot (k - r \cdot d_A)) \bmod n$, 若 $s = 0$ 则返回步骤 (3);
- (7) 将 r, s 的数据类型转换为字节串, 消息 M 的签名为 (r, s) 。

为了帮助读者理解, SM2 标准也给出了数字签名生成算法流程, 具体如图 7-3 所示。

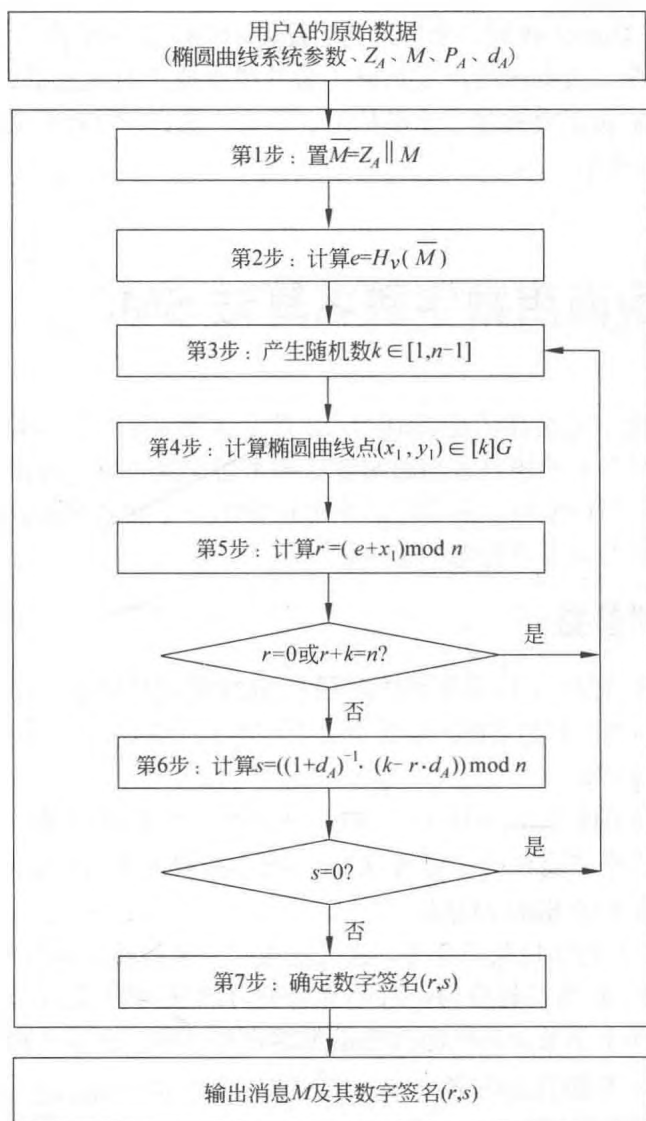


图 7-3 数字签名生成算法流程

7.6.3 验证过程

为了检验收到的消息 M' 及其数字签名 (r', s') ，作为验证者的用户 B 应实现以下运算步骤：

- (1) 检验 $r' \in [1, n-1]$ 是否成立，若不成立则验证不通过；
- (2) 检验 $s' \in [1, n-1]$ 是否成立，若不成立则验证不通过；
- (3) 置 $\bar{M}' = Z_A \parallel M'$ ；
- (4) 计算 $e' = H_v(\bar{M}')$ ，将 e' 的数据表示为整数；
- (5) 将 r' 和 s' 的数据转换为整数，计算 $t = (r', s') \bmod n$ ，若 $t = 0$ ，则验证不通过；
- (6) 计算椭圆曲线点 $(x'_1, y'_1) = [s']G + [t]P_A$ ；
- (7) 将 x'_1 的数据转换为整数，计算 $R = (e' + x'_1) \bmod n$ ，检验 $R = r'$ 是否成立，若成

立则验证通过；否则验证不通过。

为了帮助读者理解，SM2 标准也给出了数字签名生成算法流程，具体如图 7-4 所示。

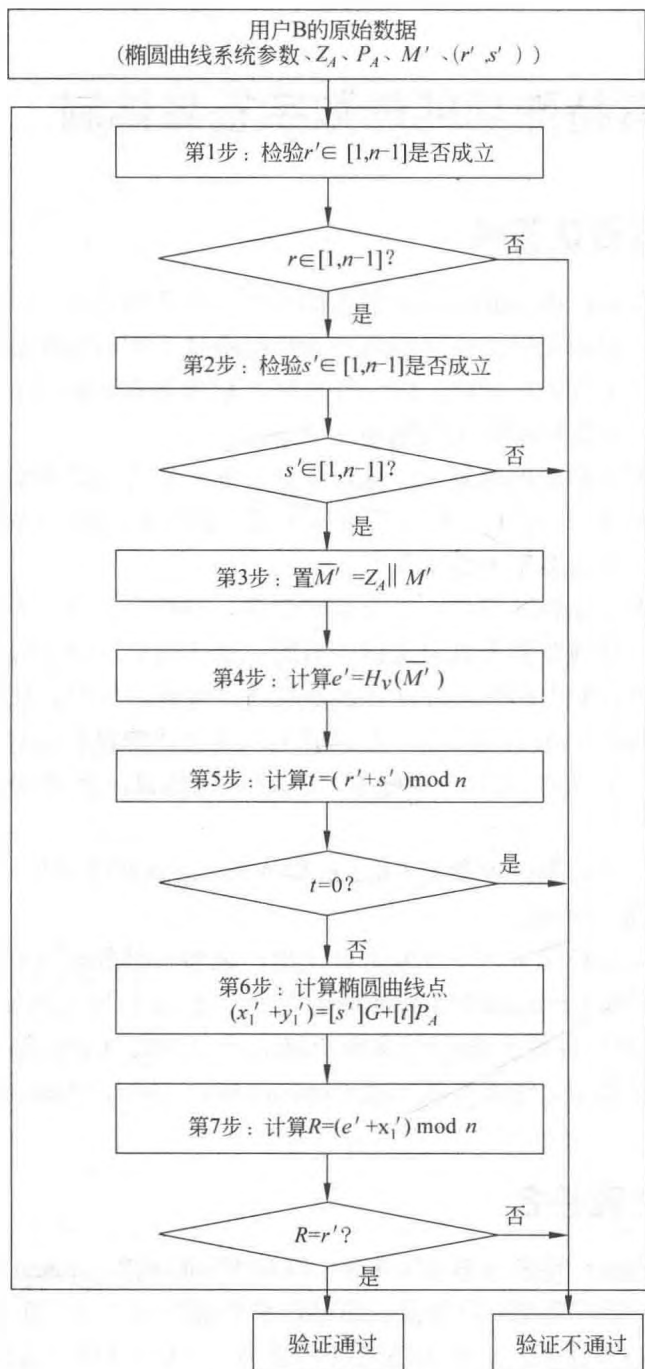


图 7-4 数字签名验证算法流程

7.6.4 签名实例

为了推广应用 SM2 算法，中国国家密码管理局在颁布 SM2 公钥密码算法时，分别

给出了 SM2 数字签名算法在两类椭圆曲线上消息签名和验证的实例,以及各步骤中的详细值。感兴趣的读者可从 Web 地址 http://www.oscca.gov.cn/News/201012/News_1197.htm 获取相关实例信息。

7.7

具有特殊功能的数字签名体制

7.7.1 不可否认签名

1989 年,由 Chaum 和 Antwerpen 引入的不可否认签名具有一些特殊性质,非常适用于某些应用。其中最本质的是在无签名者合作的条件下不可能验证签名,从而可以防止复制或散布他所签文件的可能性,这一性质使产权拥有者可以控制产品的散发。这在电子出版物的知识产权保护中将大有用场。

普通数字签名可以精确地被复制,这对于如公开声明之类文件的散发是必需的,但对另一些文件如个人或公司信件,特别是有价值文件的签名,如果也可随意复制和散发,就会造成灾难。这时就需要不可否认签名。

在签名者合作下才能验证签名,这会给签名者一种机会,即在不利于他时,他可以拒绝合作,以达到否认他曾签署过此文件的目的。为了防止此类事件发生,不可否认签名除了采用一般签名体制中的签名算法和验证算法(或协议)外,还需要第 3 个组成部分,即否认协议(Disavowal Protocol),签名者可利用否认协议向法庭或公众证明一个伪造的签名确实是假的;如果签名者拒绝参与执行否认协议,就表明签名确实是由他签署的。

有关不可否认签名体制,可参考 Chaum 和 Antwerpen 的文章[Chaum 等 1989]和王育民等的书[王育民等 1999]。

不可否认签名可以和秘密共享体制组合使用,成为一种分布式可变换不可否认签名(Distributed Convertible Undeniable Signature),它由一组人中的几个人参与协议执行来验证某人的签名。相关内容可参阅相关文献[Pederson 1991; Harn 等 1992; Sakano 等 1993]。有关不可否认签名,还可参阅文献[Chaum 1991, 1995; Okamoto 等 1994; Boyar 等 1991]。

7.7.2 防失败签名

防失败(Fail Stop)签名由 B.Pfitzmann 和 M.Waidner[Pfitzmann 等 1991]引入。这是一种强化安全性的数字签名,可防范有充足计算资源的攻击者。当 A 的签名受到攻击,甚至分析出 A 的密钥条件下,也难于伪造 A 的签名, A 也难以对自己的签名进行抵赖。

防失败签名体制可参考 van Heyst 和 Pederson[van Heyst 等 1992]所提方案。它是一种一次性签名方案。即给定密钥只能签署一个消息。它由签名、验证和“对伪造的证明”(Proof of Forgery)算法等 3 部分组成。

有关防失败签名体制还可参阅相关文献[Pfitzmann 等 1991; Damgård 等 1997]。

7.7.3 盲签名

对于一般的数字签名来说, 签名者总是要先知道文件内容而后才签署, 这正是通常所需要的。但有时需要某人对一个文件签名, 但又不让他知道文件内容, 把这种签名称为盲签名 (Blind Signature)。盲签名的概念是由 Chaum[Chaum 1983]最先提出的, 在选举投票和数字货币协议中将会碰到这类要求。利用盲变换可以实现盲签名, 如图 7-5 所示。

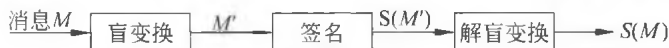


图 7-5 盲签名框图

任何盲签名, 都必须利用分割-选择原则。Chaum 提出一种更复杂的算法来实现盲签名。后来他还提出了一些更复杂但更灵活的盲签名法。

有关盲签名的各种方案可参阅相关文献[Camenisch 等 1994; Horster 等 1995; Stadler 等 1995]。盲签名在新型电子商务系统中将有重要应用[Chaum 等 1989, 1990; Chaum 1989; Okamoto 1995]。

7.7.4 群签名

群体密码学 (Group Oriented Cryptography) 由 Desmedt 于 1987 年提出。它是研究面向社团或群体中所有成员需要的密码体制。在群体密码中, 有一个公用的公钥, 群体外面的人可以用它向群体发送加密消息, 密文收到后, 由群体内部成员的子集共同进行解密。本节介绍群体密码学中有关签名的一些内容。

群签名 (Group Signature) 是面向群体密码学的一个课题, 1991 年由 Chaum 和 van Heyst 提出。它有下列几个特点: 只有群中成员能代表群体签名; 接收到签名的人可以用公钥验证群签名, 但不可能知道由群体中哪个成员所签; 发生争议时, 由群体中的成员或可信赖机构识别群签名的签名者。

例如, 这类签名可用于项目投标。所有公司应邀参加投标, 这些公司组成一个群体, 且每个公司都匿名地采用群签名对自己的标书签名。当选中了一个满意的标书后, 招标方就可识别出签名的公司, 而其他标书仍保持匿名。中标者若想反悔已无济于事, 因为在没有他参加下仍可以正确识别出他的签名。这类签名还可在其他类似场合使用。

群签名也可以由可信赖的中心协助执行, 中心掌握各签名人与所签名之间的相关信息, 并为签名人匿名签名保密; 有争执时, 可以由签名识别出签名人[Chaum 1991]。

Chaum 和 Heyst[Chaum 等 1990]曾提出 4 种群签名方案。其中, 有的由可信赖中心协助实现群签名功能, 有的采用不可否认并结合否认协议实现。

Chaum 所提方案, 不仅可由群体中一个成员的子集一起识别签名者, 还可允许群体在不改变原有系统各密钥下添加新的成员。

群签名目标是对签名者实现无条件匿名保护, 且又能防止签名者的抵赖, 因此称其为群体内成员的匿名签名 (Anonymity Signature) 更合适些[Chen 1994; Chen 等 1994]。

前面已介绍过不可抵赖签名，这里介绍在一个群体中由多个人签署文件时能实现不可抵赖特性的签名问题。Desmedt 等提出的实现方案多依赖于门限公钥体制。

一个面向群体的 (t, n) 不可抵赖签名，其中 t 是阈值， n 是群体中成员总数，群体有一公用公钥。签名时也必须要有 t 人参与才能产生一个合法的签名，而在验证签名时也必须至少有群体内成员合作参与下才能证实签名的合法性。这是一种集体签名共同负责制。L.Harn 和 S.Yang[Harn 等 1992]提出了一种 $t=1$ 和 $t=n$ 的方案。D.Wang[Wang 1996]给出了 $1 \leq t \leq n$ 的两种方案。

7.7.5 代理签名

代理(proxy)签名是某人授权其代理进行的签名。在不将其签名密钥交给代理人的条件下，如何实现委托签名呢？Mambo 等[1995]提出了一种解决办法，能够使代理签名具有如下特点：

- (1) 不可区分性(distinguishability)，代理签名与某人通常签名不可区分。
- (2) 不可伪造性(unforgeability)，只有原来签名人和所托付的代理签名人可以建立合法的委托签名。
- (3) 代理签名的差异(deviation)，代理签名者不可能制造一个合法代理签名不被检测出它是一个代理签名。
- (4) 可证实性(verifiability)，签名验证人可以相信委托签名就是原签名人认可的签名消息。
- (5) 可识别性(identifiability)，原签名人可以从委托签名确定出代理其签名人的身份。
- (6) 不可抵赖性(undeniability)，代理签名人不能抵赖他所建立的已被接受的委托签名。

有时可能需要更强的可识别性，即任何人可以从委托签名确定出代理签名人的身份。有关具体实现算法可参阅相关文献[Mambo 等 1995]。

7.7.6 指定证实人的签名

一个机构中指定一个人负责证实所有人的签名，任何成员所签的文件都具有不可否认性，但证实工作均由指定人完成，这种签名称做指定证实人的签名(Designated Confirmer Signatures)，它是普通数字签名和不可否认数字签名的折中。签名人必须限定由谁才能证实他的签名；但是，如果让签名人完全控制签名的实施，他可能会用肯定或否定方式拒绝合作，他也可能为此宣布密钥丢失，或可能根本不提供签名。指定证实人签名，可以给签名人一种不可否认签名的保护，但又不会让他滥用这类保护。这种签名也有助于防止签名失效，例如，在签名人的签名密钥确实丢失，或在他休假、病倒甚至已去世时都能对其签名提供保护。

指定证实人的签名可以用公钥体制结合适当的协议设计来实现。证实人相当于仲裁角色，他将自己的公钥公开，任何人对某文件的签名都可以通过他来证实。有关具体算

法可参阅相关文献[Okamoto 等 1994]。

7.7.7 一次性数字签名

若数字签名机构至多只能对一个消息进行签名, 否则签名就可被伪造, 这种签名被称做一次性 (One Time) 签名体制。在公钥签名体制中, 它要求对每个消息都要用一个新的公钥作为验证参数。一次性数字签名的优点是产生和证实都较快, 特别适用于要求计算复杂性低的芯片卡。有关一次性数字签名, 人们已提出几种实现方案, 如 Rabin 一次性签名方案、Merkle 一次性签名方案、GMR 一次性签名方案、Bos 等的一次性签名方案。这类方案多与可信赖第三方相结合, 并通过认证树结构实现[Menezes 等 1997]。

7.7.8 双有理签名方案

Shamir 在 1993 年提出了双有理签名方案[Shamir 1993], Coppersmith 对其进行了攻击和分析研究, 具体内容可参见相关文献[Coppersmith 等 1997]。

7.8 数字签名的应用

数字签名方案常因不同的应用而异, 本文将在后续章节中介绍数字签名在协议中的各种各样的应用, 如数字签名应用在认证的密钥建立协议、数字证书与公钥基础设施等应用场景中。

习 题

一、填空题

1. 类似手书签名, 数字签名也应满足_____、_____、_____和_____。
2. 按明、密文的对应关系划分, 数字签名可以分为_____和_____。
3. RSA 签名体制的安全性依赖于_____。
4. ElGamal 签名体制的安全性依赖于_____。
5. 不可否认签名的本质是_____。
6. 群签名是面向_____, 其目的是_____。
7. SM2 是国家密码管理局于 2010 年颁布的基于_____密码算法, 具体包括两个算法 1 个协议, 分别是_____、_____和_____。

二、思考题

1. 分析 RSA 算法存在的安全缺陷。
2. 查找并阅读 SM2 椭圆曲线公钥密码算法标准, 了解算法流程, 构思一种 SM2 数字签名算法的应用场景。
3. 比较 ElGamal 签名体制与 SM2 签名算法的异同。

4. 比较 RSA 签名体制、ElGamal 签名体制和 Schnorr 签名体制的异同。
5. 比较签名标准算法 DSA 与 ElGamal 签名体制的异同。
6. 编程实现 SM2 椭圆曲线数字签名方案。
7. 什么是不可否认签名、防失败签名、盲签名和群签名？
8. 当签名与加密结合时，有两种顺序：一种是先加密后签名，另一种是先签名后加密，试比较这两种方式的安全性。